Numerical Computing

Lecture 8: Eigenvalue Problems

Francisco Richter Mendoza

Università della Svizzera Italiana
Faculty of Informatics, Lugano, Switzerland

June 22, 2025

Lecture Overview

- **Mathematical Foundation**: Definitions, properties, sensitivity.
- **Power Method and Variants**: PageRank, Inverse Iteration, Rayleigh Quotient.
- **Singular Value Decomposition (SVD)**: Applications in compression and LSA.
- **General Methods**: QR Algorithm, Hessenberg reduction, SVD computation.

The Eigenvalue Problem

Definition

For a matrix $A \in \mathbb{R}^{n \times n}$, find scalars λ (eigenvalues) and nonzero vectors x (eigenvectors) such that:

$$Ax = \lambda x$$

- Equivalent to finding roots of the characteristic polynomial: $p(\lambda) = \det(A - \lambda I) = 0$.
- This is a polynomial of degree n, so A has exactly n eigenvalues (counting multiplicities) in $\mathbb{C}$.

Spectral Properties

Definitions

- **Spectrum** $\sigma(A)$: The set of all eigenvalues $\{\lambda : \det(A - \lambda I) = 0\}$.
- **Spectral Radius** $\rho(A) = \max_{\lambda \in \sigma(A)} |\lambda|$.
- **Algebraic Multiplicity**: Multiplicity of λ as a root of $p(\lambda)$.
- **Geometric Multiplicity**: Dimension of the eigenspace $\dim(\ker(A - \lambda I))$.

Eigenvalue Sensitivity

- **Symmetric Matrices**: Well-conditioned. Eigenvalue errors $|\Delta\lambda| \approx \epsilon$.
- **Non-Normal Matrices**: Can be ill-conditioned. Errors can be orders of magnitude larger.
- Sensitivity is bounded by $\kappa(V)\epsilon$, where V is the eigenvector matrix.
- For symmetric matrices, V is orthogonal $\implies \kappa(V) = 1$.

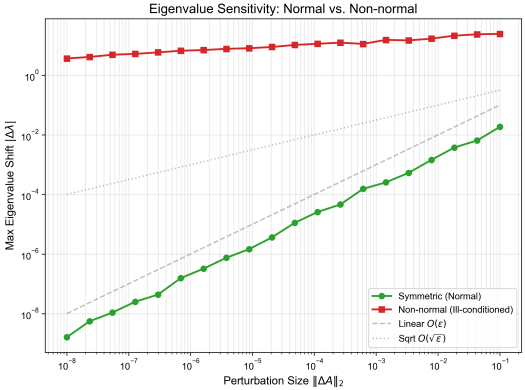


Figure: Sensitivity: Symmetric vs. Non-Normal

Special Matrix Classes

Theorem: Symmetric Matrices ($A = A^T$)

- All eigenvalues are real.
- Eigenvectors corresponding to distinct eigenvalues are orthogonal.
- A is diagonalizable by an orthogonal matrix: $A = Q\Lambda Q^T$.

Theorem: Normal Matrices ($A^T A = AA^T$)

- Includes symmetric, skew-symmetric, and orthogonal matrices.
- Unitarily diagonalizable: $A = Q\Lambda Q^*$.
- $\|A\|_2 = \rho(A)$.

The Power Method

Goal

Find the dominant eigenvalue λ_1 (largest in magnitude) and its eigenvector.

Algorithm

1. Start with random vector $v^{(0)}$.
2. Iterate: $w = Av^{(k-1)}$, $v^{(k)} = w/\|w\|$.
3. $\lambda^{(k)} = (v^{(k)})^T A v^{(k)}$ (Rayleigh Quotient).

- Convergence rate depends on the spectral gap $|\lambda_2/\lambda_1|$.

Power Method Convergence

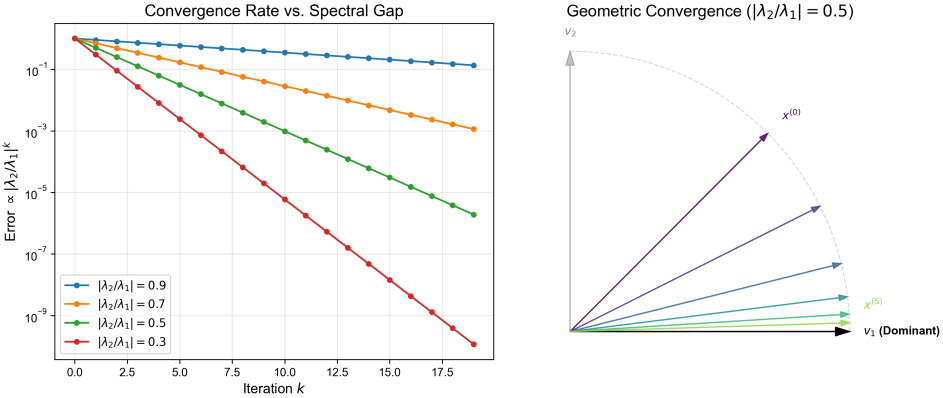


Figure: Left: Linear error decay determined by spectral gap. Right: Geometric intuition showing rotation towards dominant eigenvector.

PageRank: The Random Surfer Model

The Process

Imagine a "random surfer" clicking links on the web.

- Let $x^{(t)}$ be a probability vector where $x_i^{(t)}$ is the probability the surfer is on page i at step t.
- Transition Matrix A: A_{ij} is the probability of moving from page j to page i.
- **Dynamics**: In one step, the probability distribution evolves as:

$$x^{(t+1)} = Ax^{(t)}$$

Why the Dominant Eigenvalue?

The Steady State

We care about the **long-term behavior**. Where does the surfer spend their time? We look for a **steady state** x where the probabilities stop changing:

$$x^{(t+1)} = x^{(t)} \implies \mathrm{x} = A\mathrm{x}$$

The Connection

The equation $\mathrm{x} = A\mathrm{x}$ is exactly the eigenvalue problem $A\mathrm{x} = \lambda\mathrm{x}$ with $\lambda = 1$.

- For column-stochastic matrices, the largest eigenvalue is always $\lambda_1 = 1$.
- All other eigenvalues have $|\lambda_i| < 1$ (or ≤ 1).
- **Conclusion**: As $t \to \infty$, contributions from $\lambda_i < 1$ decay to zero. The system *must* converge to the dominant eigenvector.
- **Rank = Steady State Probability**.

PageRank Example

- Toy network with 6 nodes.
- Matrix A constructed from outlinks.
- $A_{ij} = 1/N_j$ if $j \to i$.
- Power method converges to steady-state probability.

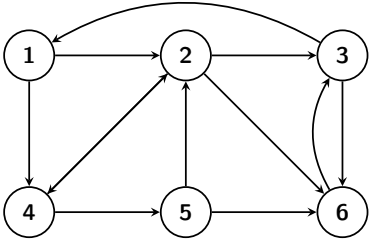


Figure: Toy Network

Inverse Iteration

Shift and Invert Technique

- If $\lambda_2 \approx \lambda_1$, power method is slow.
- Apply power method to $(A - \alpha I)^{-1}$.
- Eigenvalues are $\mu_j = \frac{1}{\lambda_j - \alpha}$.
- Converges to the eigenvalue closest to shift α.

Algorithm

1. Solve $(A - \alpha I)w = v^{(k-1)}$.
2. Normalize $v^{(k)} = w/\|w\|$.

- **Cost**: Requires solving a linear system at each step.

Rayleigh Quotient Iteration

Dynamic Shift

- Update the shift α at each step using the best current estimate.
- $\lambda^{(k)} = \rho(v^{(k)}) = \frac{(v^{(k)})^T A v^{(k)}}{(v^{(k)})^T v^{(k)}}$.

Convergence

- Convergence speed increases as we get closer to the eigenvalue.
- **Cubic convergence** for symmetric matrices!
- Very fast, but expensive per step (matrix refactorization).

Singular Value Decomposition (SVD)

Theorem: Existence of SVD

For **any** matrix $A \in \mathbb{R}^{m \times n}$, there exists a factorization:

$$A = U \Sigma V^T$$

where:

- $U \in \mathbb{R}^{m \times m}$ is orthogonal ($U^T U = I$). Columns are **left singular vectors**.
- $V \in \mathbb{R}^{n \times n}$ is orthogonal ($V^T V = I$). Columns are **right singular vectors**.
- $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. These are the **singular values**.

- **Geometric Interpretation**: Every linear transformation is a rotation (V^T), followed by a scaling (Σ), followed by another rotation (U).

Solving Almost Singular Systems

- If A is ill-conditioned, standard solvers (Gaussian elimination) fail.
- **Regularization**: Replace problem with a nearby well-conditioned one.
- **Truncated SVD**:
  1. Compute SVD: $A = U\Sigma V^T$.
  2. Keep only r largest singular values $(\sigma_1 \dots \sigma_r)$.
  3. Solution $x = \sum_{i=1}^{r} \frac{u_i^T b}{\sigma_i} v_i$.
- **Theorem**: This is the best rank-r approximation in ℓ_2-norm.

Example: Solving a Singular System

Example

$$A = \begin{pmatrix} 1 & 1 \\ 3 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 6 \end{pmatrix}$$

- A is singular. Gaussian elimination fails.
- SVD finds effective rank $r = 1$.
- Computes minimum norm solution $x = (1, 1)^T$.
- Stable even with perturbations.

Image Compression via SVD (1/2)

Idea

Approximating a matrix A with its first k singular terms:

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$$

captures the most energy/information with minimal storage.

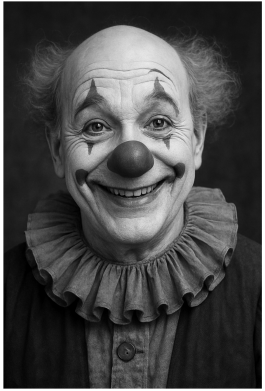
Rank-20 Approximation

Using only $r = 20$ singular values (10% of min dimension) retains the main features.
Storage: $20 \times (200 + 320 + 1) \approx 10,000$ vs $64,000$ original.

Image Compression via SVD (2/2)

Original (200×320)

Original Image (1536x1024)



Compressed (Rank-20)

Rank-20 Approximation

Latent Semantic Analysis (LSA) - Concept

The Problem

- **Synonymy**: Different words describe the same concept (e.g., "car" vs. "automobile").
- **Polysemy**: Same word has multiple meanings (e.g., "bank" of river vs. financial "bank").
- Simple keyword matching fails to capture these relationships.

The Solution: Term-Document Matrix

Construct a matrix $A \in \mathbb{R}^{n \times m}$:

- n rows = terms (words).
- m columns = documents.
- A_{ij} = frequency (or weight) of term i in document j.

LSA Example: Documents and Vocabulary

Toy Data Set

Documents:

- D1: "Numerical computations are fun."
- D2: "Numerical algorithms and numerical methods are interesting."

Vocabulary

After removing stop words, we have 6 unique terms:

1. algorithms
2. computations
3. fun
4. interesting
5. methods
6. numerical

LSA Example: Matrix and Query

Term-Document Matrix A:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix} \begin{array}{c} \leftarrow \text{algorithms} \\ \leftarrow \text{computations} \\ \leftarrow \text{fun} \\ \leftarrow \text{interesting} \\ \leftarrow \text{methods} \\ \leftarrow \text{numerical} \end{array}$$

Query:
"numerical methods"

$$\mathrm{q} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

LSA: The Math

Truncated SVD Projection

Compute SVD $A = U\Sigma V^T$ and keep largest k singular values:

$$A_k = U_k \Sigma_k V_k^T$$

This projects terms and documents into a k-dimensional "semantic space" where noise is reduced.

Similarity Measure

To find relevant documents, compute the cosine similarity between the query q and document vectors d_j (columns of A):

$$\cos(\theta_j) = \frac{\mathrm{q}^T \mathrm{d}_j}{\|\mathrm{q}\| \|\mathrm{d}_j\|}$$

In the reduced space, we compare projected vectors to find conceptual matches.

Connection to Modern AI: From LSA to Transformers

Embeddings

LSA pioneered mapping words to vectors where "closeness" $\approx$ similarity.

From Dot Product to Attention

LSA uses $\mathrm{q}^T\mathrm{d}$ for relevance. Transformers use QK^T:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- q (Single Query) $\longleftrightarrow$ Q (Matrix of Queries).
- d (Single Document) $\longleftrightarrow$ K (Matrix of Keys).
- QK^T computes **all** pairwise dot products at once.

Evolution: LSA is static/linear. Transformers are dynamic/non-linear. But **dot product similarity** remains the foundation.

Rank Deficient Linear Least Squares

Problem

$\min_{\mathrm{x}} \|\mathrm{b} - A\mathrm{x}\|_2$ where A is rank-deficient. Infinite solutions exist.

Minimum Norm Solution

We seek the unique solution with minimum $\|\mathrm{x}\|_2$.

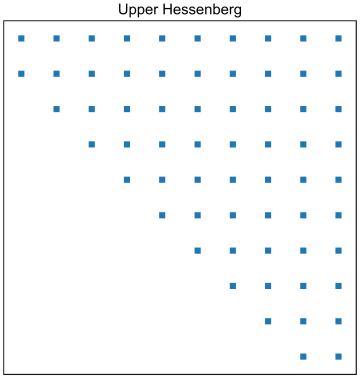
$$\mathrm{x} = A^{\dagger}\mathrm{b} = V\Sigma^{\dagger}U^{T}\mathrm{b}$$

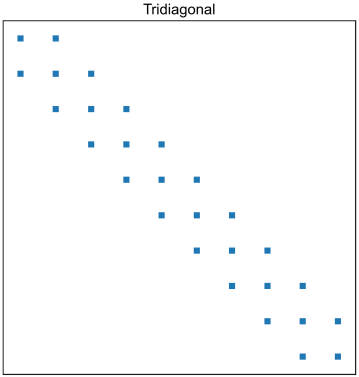
where $\Sigma^{\dagger}$ inverts non-zero singular values and leaves zeros as zeros.

The Two-Stage Approach

Modern algorithms (QR, SVD) typically work in two stages:

1. **Finite Reduction**: Reduce A to condensed form (Hessenberg/Tridiagonal) using orthogonal transformations. Preserves eigenvalues.
2. **Iterative Convergence**: Apply iterative method (QR) to condensed form. Cheap per step ($O(n^2)$ or $O(n)$).





The QR Algorithm

Stage 1: Hessenberg Reduction

- Use Householder reflectors to zero out entries below subdiagonal.
- Similarity transformation: $H = Q^T A Q$.

Stage 2: QR Iteration with Shifts

- Iterate: $A_k - \mu_k I = Q_k R_k \implies A_{k+1} = R_k Q_k + \mu_k I$.
- Converges to Schur form (Upper Triangular).
- **Shifts** (e.g., Wilkinson) accelerate convergence to cubic.

Householder Reflectors: The Building Block

Definition

A **Householder reflector** is an orthogonal matrix:

$$H = I - 2\frac{vv^T}{v^T v}$$

where v is a carefully chosen vector.

Properties:

- Orthogonal: $H^T H = I$
- Symmetric: $H^T = H$
- Reflects across hyperplane $\perp v$

Key use:

- Choose v to zero out selected entries
- Numerically stable
- Cost: $O(n)$ per reflection

Example: Transform $x = [3, 4, 0]^T$ to $[\pm 5, 0, 0]^T$ by reflecting across the right hyperplane.

Computing the SVD (1/2): Bidiagonal Reduction

Goal: Reduce A to bidiagonal form B

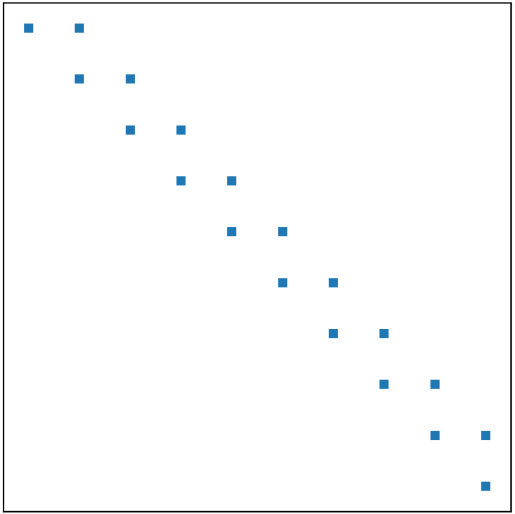
Method:

- Apply Householder from **left** to zero below diagonal
- Apply Householder from **right** to zero above super-diagonal

Result: Matrix B with entries only on diagonal and first super-diagonal

Cost: $O(mn^2)$ operations

Bidiagonal



Computing the SVD (2/2): Iterative Diagonalization

Iterative diagonalization

Direct solution is impossible (Abel-Ruffini). We use an iterative QR-like method on $B^T B$.

- Each iteration drives the super-diagonal of B to zero.
- The diagonal entries converge to the singular values σ_i.
- Cost per iteration is $O(n)$ because B is bidiagonal.

Key Takeaways

- SVD is computed in two stages: finite reduction to bidiagonal form, then iterative diagonalization.
- Dominant singular values capture the most important directions/energy.
- Applications: image compression, Latent Semantic Analysis, and the attention mechanism in Transformers.
- Iterative step is cheap and scales to large matrices.

References

- Ascher, U. M., & Greif, C. (2011). *A First Course in Numerical Methods*. SIAM.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press.
- Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.